Inheritance and polymorphism in C++

BIE-PA2 – Programming and Algorithmic II

Ladislav Vagner

Department of Theoretical Computer Science
Faculty of Information Technology
Czech Technical University in Prague
xvagner@fit.cvut.cz

March 30, 2026

Overview

- Polymorphism.
- Inheritance.
- Static and dynamic binding.

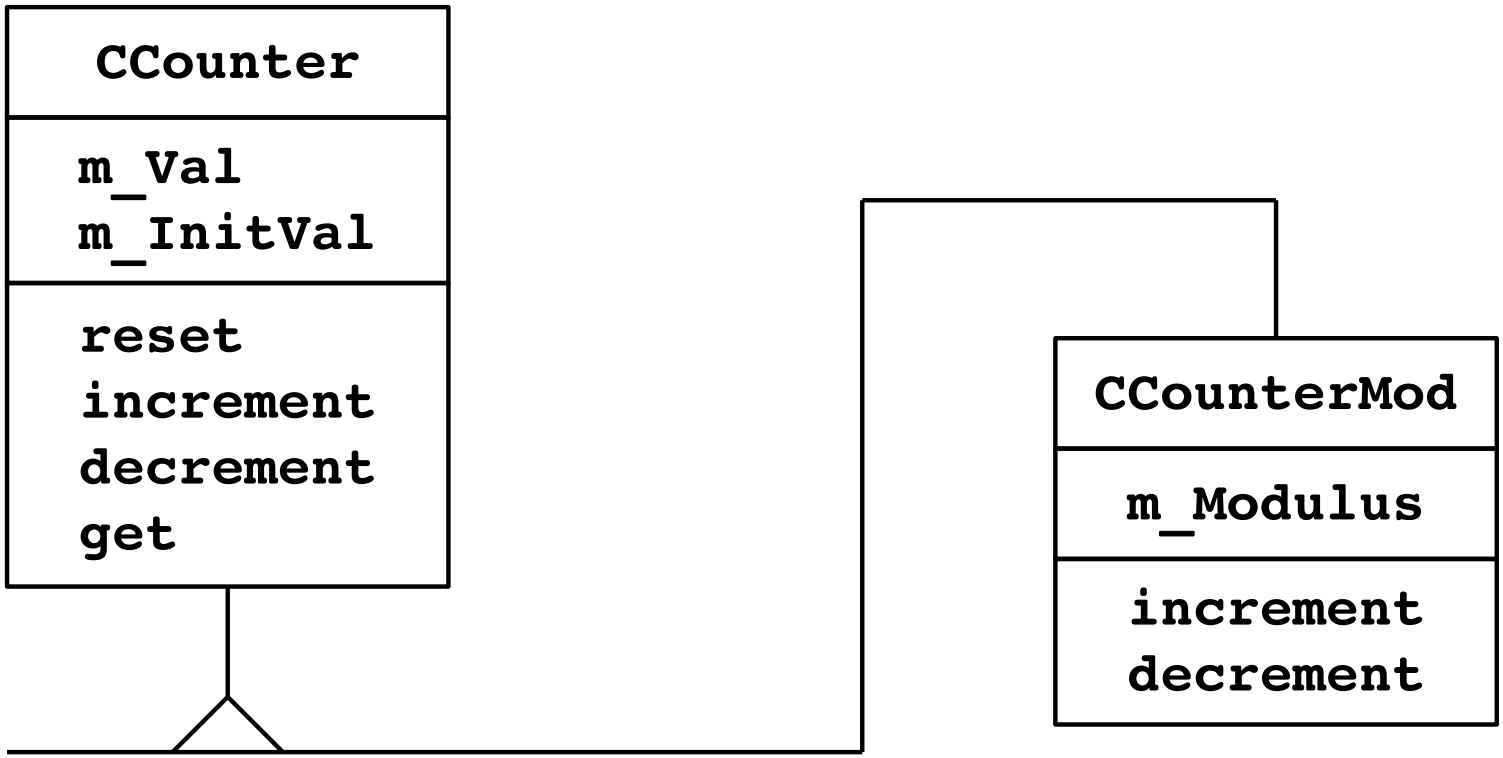
Polymorphism

- Polymorphism has many meanings like OOP itself.
- For the purposes of PA2, we mean run-time polymorphism implemented using C++ tools, specifically:
  - We have an interface consisting of virtual methods of an (abstract) class.
  - We work with objects of this class using pointers or references, so they can be implemented by subclasses.
  - We call methods without checking the actual type of the objects.
  - The called methods have sufficiently different implementations in the subclasses, so that the whole thing makes sense (otherwise it's not polymorphism but a bad design).
- Templates are sometimes considered compile-time polymorphism.

Inheritance

- Problem – develop a class representing a counter (counting integer numbers). The counter shall count modulus some number `m`.
- We already developed integer counter – class `CCounter`.
- The new class only improves/extends the function of the already developed class.
- The new class may be derived from the existing class by the means of inheritance:
  - all existing member variables will be inherited,
  - new member variables may be introduced,
  - existing methods may be inherited, or their function may be overridden,
  - new methods may be introduced.

Inheritance



Inheritance

```
// counter.h
class CCounter {
protected:
  int m_Val;
  int m_InitVal;
public:
  void increment()  { m_Val++; }
  void decrement()  { m_Val--; }
  void reset()      { m_Val = m_InitVal; }
  int get() const   { return m_Val; }
  CCounter(int val) { m_InitVal = val; reset(); }
};
```

- `protected` keyword is used to restrict access to a member variable / method to the class itself and the classes derived from the class.

Inheritance

```
// countermod.h
#include "counter.h"
class CCounterMod : public CCounter {
protected:
  int m_Modulus;
public:
  void increment();
  void decrement();
  CCounterMod(int val, int modulus);
};
```

- Colon in the class declaration separates class name and the list of the class ancestors.

Inheritance

```
// countermod.cpp
#include "countermod.h"
CCounterMod::CCounterMod(int val, int mod) :
  CCounter(val % mod) {
  // call constructor from the ancestor
    m_Modulus = mod;
}

void CCounterMod::increment() {
   CCounter::increment();
   // call method increment() from the ancestor
   m_Val = m_Val % m_Modulus;
}
void CCounterMod::decrement() {
   CCounter::decrement();
   m_Val = m_Val % m_Modulus;
}
```

Using CCounterMod

```
#include "countermod.h"
#include <iostream>
using namespace std;
int menu() { }
// displays menu, ret 0 = quit, 1,2,3 = counter manipulation

int main() {
  CCounterMod cnt(0, 5);
  for (;;) {
    cout << "Val = " << cnt.get() << endl;
    switch (menu()) {
      case 1: cnt.increment(); break;
      case 2: cnt.decrement(); break;
      case 3: cnt.reset(); break;
      case 0: return(0);
    }
  }
}
```

Derived classes

- Derived (inherited) classes have the following syntax:

```
class T1 : public T {
  // new member variables (declarations)
  // new methods (declarations)
  // overridden methods (declarations)
};
```

- Member variables in the derived class:
  - new member variables may be introduced,
  - existing member variables are inherited,
  - existing member variables cannot be removed,
  - data type of the existing variables cannot be changed.
- Methods in the derived class:
  - new methods may be introduced,
  - existing methods are inherited,
  - existing methods may be overridden (same signature, different implementation).

Derived classes

- Preserve visibility:

```
class T1 : public T { ... };
```

  - member variables/methods inherited from `T` have the same visibility in `T1`.

- Make all inherited methods/member variables `private`:

```
class T1 : private T { ... };
```

  - inherited member variables / methods are not visible outside `T1`,
  - effectively achieves inheritance but suppresses polymorphism.

- If visibility is not specified:

```
class T1 : T { ... };
//  class T1 : private T { ... ; }

struct T1 : T { ... };
//  struct T1 : public T { ... ; }
```

Derived classes and assign operator

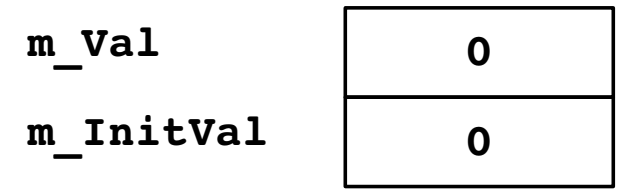
- An instance of a derived class may be assigned to the ancestor.
- An instance of a base class cannot be assigned to the descent.

```
class T { ... };
class T1 : public T { ... };
class T2 : public T { ... };

T x;     T1 x1;       T2 x2;
T *p;    T1 *p1;      T2 *p2;

x  = x1;  x = x2;  // both ok
x1 = x;   x1 = x2; // both invalid

p  = p1;  p  = p2; // both ok
p1 = p;   p1 = p2; // both invalid
```

Object implementation (memory layout)

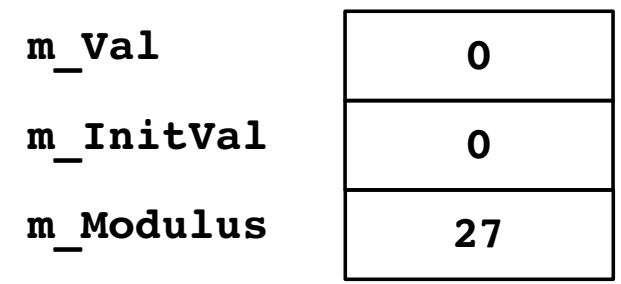
```
class CCounter {
protected:
  int m_Val;
  int m_InitVal;
public:
  ....
};
```

CCounter x(0)




```
class CCounterMod :
  public CCounter {
protected:
  int m_Modulus;
public:
  ....
};
```

CCounterMod y(0,27)



Static and dynamic binding

```
CCounter    c(0);
CCounterMod cm(0,5);
...
c.increment();  // CCounter::increment()
cm.increment(); // CCounterMod::increment()

c = cm;

c.increment();
```

Static and dynamic binding

```
CCounter    c(0);
CCounterMod cm(0,5);
...
c.increment();  // CCounter::increment()
cm.increment(); // CCounterMod::increment()

c = cm;

c.increment();  // CCounter::increment()
```

- The operation is determined using the data type of the variable (c here).
- The method is chosen in compile-time.
- Static binding of methods.

Static and dynamic binding

```
CCounter    * p = new CCounter(0);
CCounterMod * pm = new CCounterMod(0,5);
...
p->increment();  // CCounter::increment()
pm->increment(); // CCounterMod::increment()

delete p;
p = pm;

p->increment();
```

Static and dynamic binding

```
CCounter    * p = new CCounter(0);
CCounterMod * pm = new CCounterMod(0,5);
...
p->increment();  // CCounter::increment()
pm->increment(); // CCounterMod::increment()

delete p;
p = pm;

p->increment();  // CCounter::increment()
```

- The operation is again determined using the data type of the variable (p here).
- The method is chosen in compile-time.
- Static binding of methods.

Static and dynamic binding

- Static binding is faster, however, it does not support polymorphism:
  - the caller must distinguish the objects and their types,
  - the caller must have the knowledge of all possible derived classes,
  - the caller (a higher level layer in the program) must care about implementation details in the objects it uses.
- Dynamic binding determines the operation in run-time, based on the object it processes:
  - the execution is somewhat slower,
  - the caller processes objects (e.g. counters here) and performs some operation. It does not have to distinguish the object type to perform the operation,
  - if new subclasses are developed, existing code will know how to use them, moreover, the existing code does not need to be modified.

Static and dynamic binding

- We will develop function named `funWithCounter` which will handle the logic (menu, modifications to the counter instance).
- The function will be able to handle any object derived from `CCounter`.
- The class `CCounter` will use dynamic binding.

```
#include "counter.h"
#include <iostream>
using namespace std;

int menu() { ... }

void funWithCounter(CCounter * x);
```

Static and dynamic binding

```
void funWithCounter(CCounter * x) {
  for (;;) {
    cout << "Val = " << x->get() << endl;
    switch (menu()) {
      case 1: x->increment(); break;
      case 2: x->decrement(); break;
      case 3: x->reset(); break;
      case 0: return;
    }
  }
}
```

Static and dynamic binding

```
class CCounter {
protected:
  int m_Val;
  int m_InitVal;
public:
  virtual void increment()  { m_Val++; }
  virtual void decrement()  { m_Val--; }
  void reset()      { m_Val = m_InitVal; }
  int get() const   { return m_Val; }
  CCounter(int val) { m_InitVal = val; reset(); }
};
```

- Keyword `virtual` indicates dynamic binding of a particular method.

Static and dynamic binding

```
class CCounterMod : public CCounter {
  ...
  void increment() override {
    CCounter::increment();
    m_Val %= m_Modulus;
  }
  void decrement() override {
    CCounter::decrement();
    m_Val %= m_Modulus;
  }
  ...
};
```

- Keyword `virtual` is not needed here – the methods are automatically dynamically bound (inherited from the ancestor).
- Keyword `override` is also not needed but it is recommended.

Keyword `override` (since C++ 11)

- Programmer claims that the method overrides one in the parent,
- compiler checks that it is indeed true,
- if not, it emits an error.
- Use of `override` limits the possibility of a mistake, examples:
  - types or number of parameters don't match,
  - `const` or `noexcept` qualification differs.
- GCC with flag `-Wsuggest-override` warns about missing `override`.

```
class Base {
  ...
  virtual int m1();
  virtual void m2() const;
};
class Derived : public Base {
  ...  // error - methods do not override
  virtual long m1(int x) override;
  virtual void m2() override;
};
```

Without using override

```
class Base {
  ...
  virtual int m1();
  virtual void m2() const;
};
class Derived : public Base {
  ...
  virtual long m1(int x);
  virtual void m2();
  // New signatures, shadowing of methods
  // No warning, but probably wrong
};

Derived d;
Base& b = d;
b.m1(); // Base::m1
b.m2(); // Base::m2
```

Static and dynamic binding

```
...
int main() {
  CCounter c (0);
  CCounterMod cm(0, 24);

  cout << "CCounter" << endl;
  funWithCounter(&c);

  cout << "CCounterMod" << endl;
  funWithCounter(&cm);
  return 0;
}
```

Static and dynamic binding

- Could be the interface of `funWithCounter` changed to:

```
void funWithCounter(CCounter& x)
```

Static and dynamic binding

- Could be the interface of `funWithCounter` changed to:

  ```
  void funWithCounter(CCounter& x)
  ```

  Yes, the program will operate properly.

Static and dynamic binding

- Could be the interface of `funWithCounter` changed to:

  ```
  void funWithCounter(CCounter& x)
  ```

  Yes, the program will operate properly.
- Could be the interface of `funWithCounter` changed to:

  ```
  void funWithCounter(CCounter x)
  ```

Static and dynamic binding

- Could be the interface of `funWithCounter` changed to:

  ```
  void funWithCounter(CCounter& x)
  ```

  Yes, the program will operate properly.
- Could be the interface of `funWithCounter` changed to:

  ```
  void funWithCounter(CCounter x)
  ```

  No, the program will revert to `CCounter` operation (as if dynamic binding was not present). Why?

Static and dynamic binding

- If the object is passed via a pointer (or a reference), the code in `funWithCounter` operates with the original object:
  - it has the original instance,
  - it has the original interface.
- If the object is passed by value (a copy), then:
  - an instance of `CCounter` is created (copy constructor),
  - the new instance is an object of type `CCounter`,
  - it has the interface of `CCounter` and methods of `CCounter`.
- No surprise it behaves like a `CCounter` – it is actually a `CCounter`.
- Why the behavior is different in Java (and other languages)?

Static and dynamic binding

- If the object is passed via a pointer (or a reference), the code in `funWithCounter` operates with the original object:
  - it has the original instance,
  - it has the original interface.
- If the object is passed by value (a copy), then:
  - an instance of `CCounter` is created (copy constructor),
  - the new instance is an object of type `CCounter`,
  - it has the interface of `CCounter` and methods of `CCounter`.
- No surprise it behaves like a `CCounter` – it is actually a `CCounter`.
- Why the behavior is different in Java (and other languages)?
  - Java does not pass objects by value.
  - Java always passes references.

Static and dynamic binding

```
class C                { void foo(); }
class C1 : public C    { void foo(); }
class C2 : public C1   { void foo(); }
class C3 : public C2   { void foo(); }

C  x;  x .foo(); //  C  :: foo
C1 x1; x1.foo(); //  C1 :: foo
C2 x2; x2.foo(); //  C2 :: foo
C3 x3; x3.foo(); //  C3 :: foo

x = x1; x.foo();
```

Static and dynamic binding

```
class C                { void foo(); }
class C1 : public C    { void foo(); }
class C2 : public C1   { void foo(); }
class C3 : public C2   { void foo(); }

C  x;  x .foo(); //  C  :: foo
C1 x1; x1.foo(); //  C1 :: foo
C2 x2; x2.foo(); //  C2 :: foo
C3 x3; x3.foo(); //  C3 :: foo

x = x1; x.foo(); //  C  :: foo
x = x2; x.foo(); //  C  :: foo
x = x3; x.foo(); //  C  :: foo
```

Static binding – only the type of variable `x` is important.

Static and dynamic binding

```
class C                { void foo(); }
class C1 : public C    { void foo(); }
class C2 : public C1   { void foo(); }
class C3 : public C2   { void foo(); }

C  * p  = new C;  p ->foo(); //  C  :: foo
C1 * p1 = new C1; p1->foo(); //  C1 :: foo
C2 * p2 = new C2; p2->foo(); //  C2 :: foo
C3 * p3 = new C3; p3->foo(); //  C3 :: foo

p = p1; p->foo();
```

Static and dynamic binding

```
class C                { void foo(); }
class C1 : public C    { void foo(); }
class C2 : public C1   { void foo(); }
class C3 : public C2   { void foo(); }

C  * p  = new C;  p ->foo(); //  C  :: foo
C1 * p1 = new C1; p1->foo(); //  C1 :: foo
C2 * p2 = new C2; p2->foo(); //  C2 :: foo
C3 * p3 = new C3; p3->foo(); //  C3 :: foo

p = p1; p->foo(); //  C  :: foo
p = p2; p->foo(); //  C  :: foo
p = p3; p->foo(); //  C  :: foo
```

Static binding – only the type of variable `p` is important.

Static and dynamic binding

```
class C                { virtual void foo(); }
class C1 : public C    { void foo() override; }
class C2 : public C1   { void foo() override; }
class C3 : public C2   { void foo() override; }

C  x;  x .foo(); //  C  :: foo
C1 x1; x1.foo(); //  C1 :: foo
C2 x2; x2.foo(); //  C2 :: foo
C3 x3; x3.foo(); //  C3 :: foo

x = x1; x.foo();
```

Static and dynamic binding

```
class C                { virtual void foo(); }
class C1 : public C    { void foo() override; }
class C2 : public C1   { void foo() override; }
class C3 : public C2   { void foo() override; }

C  x;  x .foo(); //  C  :: foo
C1 x1; x1.foo(); //  C1 :: foo
C2 x2; x2.foo(); //  C2 :: foo
C3 x3; x3.foo(); //  C3 :: foo

x = x1; x.foo(); //  C  :: foo
x = x2; x.foo(); //  C  :: foo
x = x3; x.foo(); //  C  :: foo
```

Dynamic binding – however `x` is always an object of type `CCounter` (with a content copied from some descendant).

Static and dynamic binding

```
class C                { virtual void foo(); }
class C1 : public C    { void foo() override; }
class C2 : public C1   { void foo() override; }
class C3 : public C2   { void foo() override; }

C  * p  = new C;  p ->foo(); //  C  :: foo
C1 * p1 = new C1; p1->foo(); //  C1 :: foo
C2 * p2 = new C2; p2->foo(); //  C2 :: foo
C3 * p3 = new C3; p3->foo(); //  C3 :: foo

delete p;
p = p1; p->foo();
```

Static and dynamic binding

```
class C                 { virtual void foo(); }
class C1 : public C     { void foo() override; }
class C2 : public C1    { void foo() override; }
class C3 : public C2    { void foo() override; }

C  * p  = new C;  p ->foo(); //  C  :: foo
C1 * p1 = new C1; p1->foo(); //  C1 :: foo
C2 * p2 = new C2; p2->foo(); //  C2 :: foo
C3 * p3 = new C3; p3->foo(); //  C3 :: foo

delete p;
p = p1; p->foo(); //  C1 :: foo
p = p2; p->foo(); //  C2 :: foo
p = p3; p->foo(); //  C3 :: foo
```

Dynamic binding, `p` is used to access different instances.

Static and dynamic binding

```
class C                 { void foo(); }
class C1 : public C     { void foo(); }
class C2 : public C1    { virtual void foo(); }
class C3 : public C2    { void foo() override; }
C  * p  = new C;  p ->foo(); //  C  :: foo
C1 * p1 = new C1; p1->foo(); //  C1 :: foo
C2 * p2 = new C2; p2->foo(); //  C2 :: foo
C3 * p3 = new C3; p3->foo(); //  C3 :: foo
delete p;
p = p1; p->foo();
```

Static and dynamic binding

```
class C                 { void foo(); }
class C1 : public C     { void foo(); }
class C2 : public C1    { virtual void foo(); }
class C3 : public C2    { void foo() override; }
C  * p  = new C;  p ->foo(); //  C  :: foo
C1 * p1 = new C1; p1->foo(); //  C1 :: foo
C2 * p2 = new C2; p2->foo(); //  C2 :: foo
C3 * p3 = new C3; p3->foo(); //  C3 :: foo
delete p;
p = p1; p->foo(); //  C :: foo
p = p2; p->foo(); //  C :: foo
p = p3; p->foo(); //  C :: foo
delete p2;
p2 = p3; p2->foo();
```

Static and dynamic binding

```
class C                { void foo(); }
class C1 : public C    { void foo(); }
class C2 : public C1   { virtual void foo(); }
class C3 : public C2   { void foo() override; }
C  * p  = new C;  p ->foo(); //  C  :: foo
C1 * p1 = new C1; p1->foo(); //  C1 :: foo
C2 * p2 = new C2; p2->foo(); //  C2 :: foo
C3 * p3 = new C3; p3->foo(); //  C3 :: foo
delete p;
p = p1; p->foo(); //  C :: foo
p = p2; p->foo(); //  C :: foo
p = p3; p->foo(); //  C :: foo
delete p2;
p2 = p3; p2->foo(); //  C3 :: foo
```

- Static binding for p and p1.
- Dynamic binding for p2 and p3.

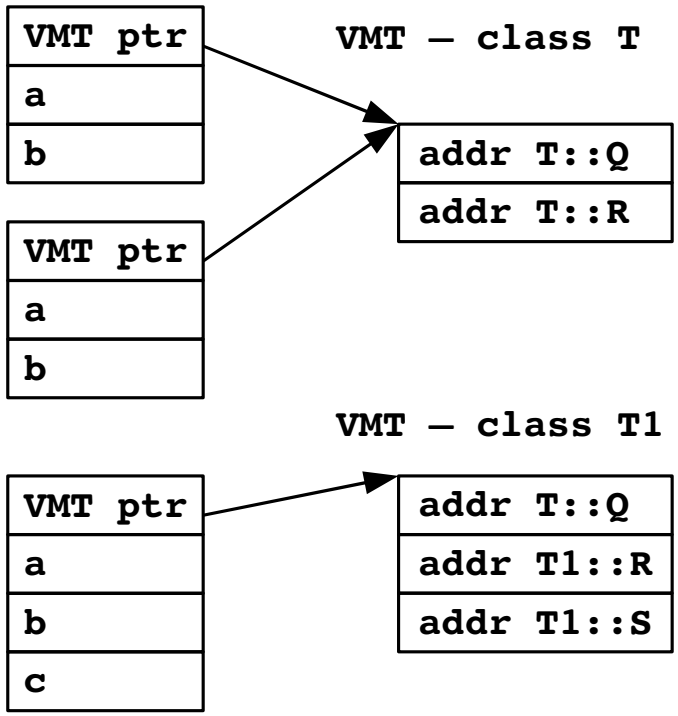
How dynamic binding works

- Dynamic binding assumes that the method is chosen in run-time:
  - objects must "know" their class to permit the search,
  - the method must be chosen very quickly (ideally in $\mathcal{O}(1)$ time).
- C++ uses VMT (virtual method table) to find the method in constant time:
  - VMT is prepared for every class, the table is generated by the compiler,
  - VMT lists addresses of all dynamically bound methods,
  - each object has a pointer to its VMT table, this pointer is initialized by the constructor,
  - methods are listed in VMT such that the name of a method corresponds to an offset in the table,
  - moreover, the name – offset mapping is preserved for all derived classes (how is it done?).
- Call to a virtual method means to index the VMT table (2x dereference) and call the code referenced by that pointer.

How dynamic binding works

```
class T {
  int a, b;
public:
  void P ();
  virtual void Q ();
  virtual void R ();
};
```

```
class T1 : public T {
  int c;
public:
  virtual void R ();
  virtual void S ();
  void U();
};
```



How dynamic binding works

- Only virtual methods are placed in the VMT.
- Method `T::Q` is not overridden, thus it is unchanged in the `T1`'s VMT.
- Method `Q` was assigned VMT index 0, `R` has index 1. These indices must be preserved in the descendant classes (e.g. in `T1`).
- New methods (e.g. `S`) are added at the end of the VMT. Method `S` will have index 2, this index will be preserved in all descendants of class `T1`.
- VMT structure is straightforward for simple inheritance. It is much more complex if multiple inheritance is used (C++ supports multiple inheritance).

Questions and answers

Questions ...